



COS 110 E03 2021 - COS 110 EO3 2021, during covid thus it was an online test

Program Design: Introduction (University of Pretoria)



Scan to open on Studocu



View Test Submission: Examination Opportunity 3

Question 1

3 out of 3 points



Write a recursive function named `productOfDigits` that returns the product of the digits of `int num`. For example, if `num` is 3114, the function should return the product of $3 * 1 * 1 * 4 = 12$. The prototype of the function is given below.

```
int productOfDigits (int  
num)
```

Selected Answer:

```
int productofDigits (int  
num) {  
    if (num < 10 ) {  
        return num; }  
    return  
    productofDigits(num/10)  
    * (n%10);  
}
```

Correct

Answer:

```
int productOfDigits(int
```





View Test Submission: Examination Opportunity 3

}

Response [None Given]

Feedback:

Question 2 1 out of 1 points



True or False: Pushing and then subsequently popping a list of elements into a stack will reverse its order.

Selected Answer: True

Correct Answer: True

Question 3 2 out of 2 points



Suppose you have a list container object called **mylist** with the following integer values.





> View Test Submission: Examination Opportunity 3 2.5 out of 4 points



Assume the class definitions below and answer the question.

```
template <class T>
struct Node {
    Node (T,
Node<T>*);
    T data ;
    Node<T> * next ;
// address of the next
element in the queue
};

template <class T>
class Queue {
private :
    Node<T> * rear; //
points to the rear;
// Circular
queue: rear->next
points to the front
public :
    Queue();
    Queue(const Queue&);
    ~Queue();
    void enqueue (T);
    T dequeue();
    bool isEmpty();
    void clear();
};
```

Write down the code for the enqueue function, assuming that the queue



Write down the code for the enqueue function, assuming that the queue is a dynamic circular queue.

Selected void

Answer: Queue<T>::enqueue
(T val) {
 Node<T> *
 newNode = NULL;
 newNode =
 new Node<T>;
 newNode->data
 = val;
 newNode =
 NULL;
 if (isEmpty())
 rear =
 newNode;
 rear->
 next = newNode;
 else {
 rear->next
 = newNode;
 rear =
 newNode;





View Test Submission: Examination Opportunity 3

Answer: int productOfDigits(int
 num)
 {

 if(num / 10 == 0) \\1
mark
 {
 return num; \\1
mark
 }
 else
 {
 return
 productOfDigits(num
 /10) * (num % 10); \\1
mark
 }

 }

Response [None Given]

Feedback:



Consider the code below:

```
#ifndef LINKEDLIST
#define LINKEDLIST
template <class T>
class LinkedList{
private :
    struct Node {
        T value;
        Node* next ;
    }

    Node* head;
public:
    LinkedList();
    LinkedList(const
LinkedList& l) ;
    ~LinkedList() ;
    void insertNode(val
n);
    void deleteNode(T
val);
    void appendNode(T
val);
    void
printLinkedList();
};
#endif
```

Is it possible to implement **deleteNode** using the head



View Test Submission: Examination Opportunity 3

Question 3 2 out of 2 points

Suppose you have a list container object called **mylist** with the following integer values.

12, 9, 8, 4, 10, 9

Write a single line of code that removes all the occurrences of 9 from the list.

Selected `mylist.remove(9);`

Answer:

Correct

Answer: `mylist.remove(9);`

Response [None Given]

Feedback:

Question 4 2 out of 2 points

Consider the code below:



New Test Submission: Examination Opportunity 3

```
var i;
```

```
void
```

```
printLinkedList();
```

```
};
```

```
#endif
```

Is it possible to implement **deleteNode** using the head pointer and only **one helper pointer**? Choose all correct answers from the given options.

Selected

Answers: No it is not possible. It is only possible if you are deleting from the head



No it is not possible. If you delete from the middle, you need to link the previous node to the next node, without overwriting the





View Test Submission: Examination Opportunity 3

current node that
you want to delete

Correct



Answers:

No it is not
possible. It is only
possible if you are
deleting from the
head



No it is not possible.
If you delete from
the middle, you
need to link the
previous node to
the next node,
without
overwriting the
address of the
current node that
you want to delete

**Question 5**

2 out of 5 points



Write a recursive function named **encodeDigits** that should take a passed-in number and return a string such that the even numbers are replaced with a "*" and the odd number are replaced with a "_". So the integer input 4321 should become "*_*_ " and the integer input 122722 should become "_**_**". The prototype of the function is given below.

`string``encodeDigits(int num)`Selected `string encodeDigits`



View Test Submission: Examination Opportunity 3

`string``encodeDigits(int num)`Selected `string encodeDigits`Answer: `(int num) {``if (num < 10) {``if (num%2 ==``0) { return "*"; }``return "_";``}``return``encodeDigits(num/10)``;``}`

Correct



Answer:

`string``encodeDigits(int num)``{``if(num==0) \ \ 1``mark``{``return "" \ \ 1`



View Test Submission: Examination Opportunity 3

```
encodeDigits(int num)
{
    if(num==0) \\ 1
    mark
    {
        return ""; \\ 1
    mark
    }
    else
    {
        if ((num % 10) %
2 == 0) \\ 1 mark
        {
            return
encodeDigits(num/10)
+ "*"; \\ 1 mark
        }
        return
encodeDigits(num /
10) + "_"; \\ 1 mark
    }
```



Question 8

2 out of 2 points



For which of the applications below will a singly linked circular list work better than a non-circular one?

Selected



Answers:

Turn order in a game



Queue data structure

Correct



Answers:

Turn order in a game



Queue data structure



> View Test Submission: Examination Opportunity 3

```
x;}
    return
    (pow(x,y))/y +
    (pow(x,y)/y+1) +
    calculateSum(x, y-1);
}
```

Correct



Answer:

```
double
calculateSum(int
x, int n)
{
    if (n==0)
return 0; //1
mark for correct
base case
// 1 + 1 +
1: for each term
below
    else return
pow(x, n)/n +
pow(x, n)/(n+1)
+
calculateSum(x,
n-1);
}
```

Response wrong if condition

Feedback:



> View Test Submission: Examination Opportunity 3

Implement a recursive function, **double calculateSum(int x, int n)**, that would calculate the above equation for a given **x** and given maximum **n**, where **n** is the power of **x**. T_n is the n th term of the summation on the right hand side. You are allowed to use mathematical library functions, but no recursive helper functions are allowed.

Selected double calculateSum

Answer:

```
(int x, int y) {
    if (y <= 1) { return
x; }
    return
(pow(x,y))/y +
(pow(x,y)/y+1) +
calculateSum(x, y-1);
}
```

Correct

Answer:

```
double
calculateSum(int
x, int n)
{
    if (n==0)
return 0; //1
mark for correct
```



```

^, f
    return
    (pow(x,y))/y +
    (pow(x,y)/y+1) +
    calculateSum(x, y-1);
}

```

Correct



Answer:

```

double
calculateSum(int
x, int n)
{
    if (n==0)
return 0; //1
mark for correct
base case
    // 1 + 1 +
1: for each term
below
    else return
pow(x, n)/n +
pow(x, n)/(n+1)
+
calculateSum(x,
n-1);
}

```

Response wrong if condition

Feedback:

```
    )) // 0.5
    mark for check
    ( can also
    check rear )
    {
        rear =
        newNode ; //
        0.5 mark for
        updating rear

        newNode->next
        = newNode ; //
        0.5 mark for
        updating next
        ( circular )
        }
    else
    {

        newNode->next
        = rear->next ;
        // 1 mark
        rear-
        >next =
        newNode ; //
        0.5 mark
        rear =
        newNode ; //
        0.5 mark
    }
}
```

Response New node

Answer:

Solution:

```
template
<class T>
void
Queue::enqueue
(T val)
{
    Node<T>*
    newNode = new
    Node (val ,
    NULL); // 0.5
    mark for init
    newNode

    // check if
    list is empty
    if(isEmpty
    ()) // 0.5
    mark for check
    ( can also
    check rear )
    {
        rear =
        newNode ; //
        0.5 mark for
        updating rear

        newNode->next
        = newNode ; //
        0.5 mark for
        updating next
        ( circular )
    }
```





> View Test Submission: Examination Opportunity 3

```
{
```

```
// Complete the code
```

```
}
```

Selected int

Answer: countEvenNumbers(Node*
head)

```
{
```

```
    Node * temp = head;
```

```
    int count = 0;
```

```
    int placeH = temp->
```

```
data;
```

```
    while (temp != NULL) {
```

```
        if (placeH%2 ==
```

```
0) {
```

```
            count++;
```

```
        }
```

```
        temp = temp->
```

```
next;
```

```
    }
```

```
    return count;
```

```
}
```

Correct

Answer: int





View Test Submission: Examination Opportunity 3
4 out of 5 points



```
class Node
{
    public :
        int data;
        Node* next;
};
```

Given the above class that define node of a linked list, complete the code of the function below number of even numbers in a linked list where the parameter **head** is a pointer to the head of the linked list.

```
int countEvenNumbers(Node* head)
{
    // Complete the code
}
```

Selected int

Answer: `countEvenNumbers(head)`





View Test Submission: Examination Opportunity 3

Correct 

Answer:

```
int
countEvenNumbers(Node*
head )
{
    Node* current = head;
\\0.5 mark
    int count = 0;
\\0.5 mark
    while (current != NULL)
\\1 mark
    {
        if (current->data %2
== 0) { \\1 mark

count++;
\\0.5 mark
        current = current-
>next;    \\ 1 mark
    }
    return count;
\\0.5 mark
}
```

Response placeH needs to change